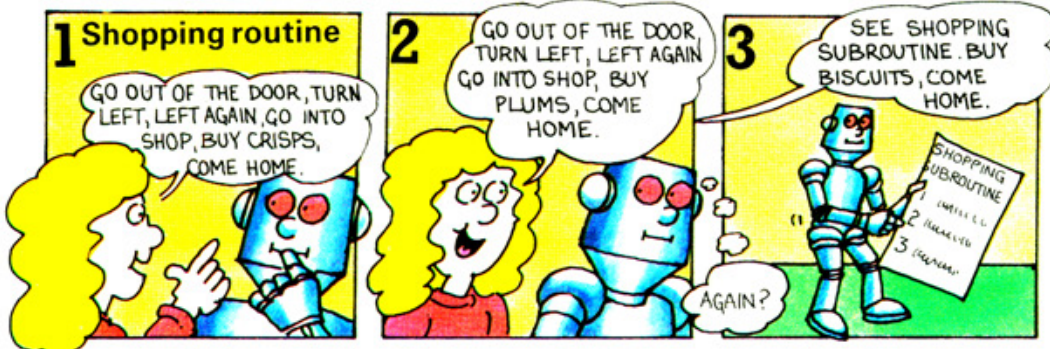


# Subroutines

A subroutine is a sort of mini-program within a program. It carries out a particular task, such as adding numbers or keeping a score, and you can send the computer to it whenever you want this task carried out. This saves writing out the program lines each time and makes the program shorter and easier to read and type into the computer.



Suppose you had a robot helper whom you could program to run errands for you. If you wanted something from the shop you would have to give it precise instructions telling it how to get there.

Each time you wanted the robot to buy something you would have to give it the same instructions. It would be much simpler to give the robot a shopping subroutine and tell it to refer to it each time.



## 4 Shopping program

```
10 PRINT "WHAT DO YOU WANT FROM THE SHOP"
20 INPUT X$
30 GOSUB 100
40 PRINT "ANYTHING ELSE"
50 INPUT M$
60 IF M$="YES" THEN GOTO 10
70 STOP
```

Line 30 sends the computer to the first line of the subroutine.

You need the word STOP at the end of the main program to stop the computer carrying on into the subroutine.



```
100 REM: SHOP SUBROUTINE
110 PRINT "GO OUT, TURN LEFT"
120 PRINT "LEFT AGAIN, ENTER SHOP"
130 PRINT "BUY ";X$;" COME HOME"
140 RETURN
```

It is useful to label a subroutine with a REM line so you know what it is for.

This sends the computer back to line 40 - the line after GOSUB.

If you forget the RETURN line you get a bug.



In BASIC, to tell the computer to go to a subroutine you use the word GOSUB with the word RETURN at the end of the subroutine. GOSUB should be followed by the number of the first line of the subroutine. RETURN does not need a line

number. The computer automatically goes back to the instruction after the one where it left the main part of the program. You can send the computer to a subroutine anywhere in the program as many times as you like.